1) What’s the difference between a class and an object?

2) What happens in memory when you write: Car car1 = new Car();

3) Why do we need a constructor in Java if we can already assign values directly to fields?

4) What happens if you don’t define any constructor in a class?

5) Can a constructor be overloaded in Java? If yes, how?

6) Can a constructor be inherited in Java? Why or why not?

7) What’s the difference between using this() and super() inside a constructor?

8) Can we use both this() and super() in the same constructor? Why or why not?

9) Can a constructor be abstract, final, or static in Java? Why or why not?

10) Can a class have private constructors? If yes, why would you use one?

11) Can a constructor call a method inside it? If yes, is there any caveat?

12) Can we overload and override methods in Java? What’s the difference between the two?

13) Can we overload the main method in Java? If yes, what happens when we run the program?

14) Can we override a static method in Java? Why or why not?

15) Can a constructor be overridden in Java? Why or why not?

16) What’s the difference between method overloading and method overriding in terms of polymorphism?

17) Do you want me to also show you a real-world example in Java where both overloading and overriding (polymorphism) are used together?

18) Can you explain the difference between compile-time polymorphism and runtime polymorphism in Java with examples?

19) Can we achieve runtime polymorphism with data members (fields) in Java?

20) In Java, can a class inherit from multiple classes? If not, why? And how do we still achieve something similar to multiple inheritance?

21) Can you explain the difference between extends and implements in Java?

22) What happens if a class implements two interfaces?

23) Can an interface extend a class in Java? Why or why not?

24) But can an interface have a reference to a class (e.g., Object methods like toString())?

25) Can you explain the difference between Abstract Classes and Interfaces, and when you would use one over the other?

26) Since Java 8 introduced default methods in interfaces, don’t they become similar to abstract classes? What’s the difference now?

27) What is encapsulation in Java, and how does it help in achieving object-oriented principles?

28) Can you give me a real-world example of where you would use it?

29) How is encapsulation different from abstraction? Would you like to try answering that?

30) Why does Java not support multiple inheritance with classes but allows it with interfaces?

31) Can you explain the difference between method overloading and method overriding in inheritance?

32) What happens if we overload a static method? And can we override a static method?

33) Can we achieve multiple inheritance using abstract classes?

34) Do you want me to also explain Polymorphism with Interfaces (like using interface Shape → Circle, Square, etc.) since that's often asked in interviews?